제 1주차

-자료구조 -

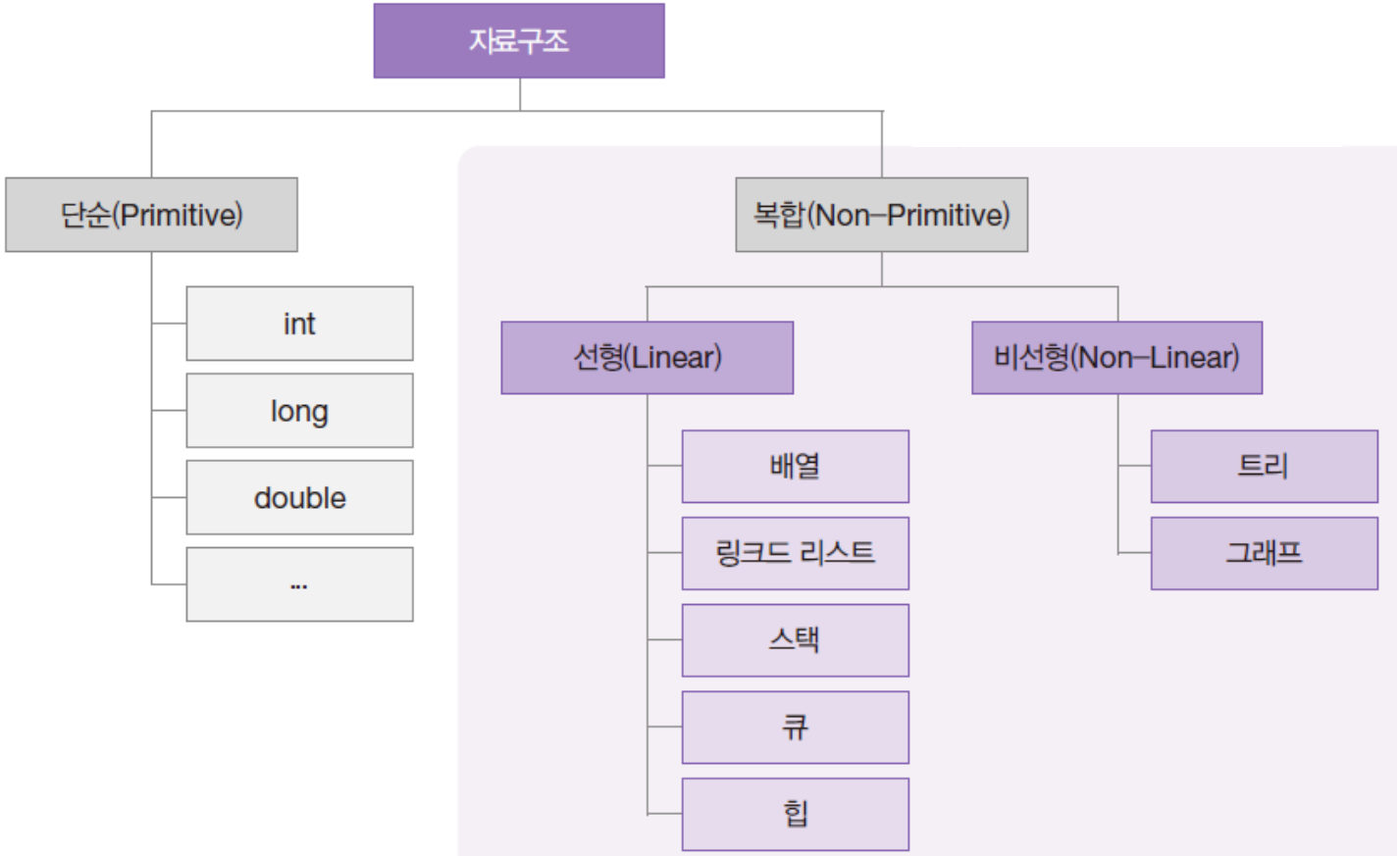
학습 목표

1. 주어진 문제를 해결하기 위한 기초적인 자료구조를 이해한다.
2. 가상 코드(Pseudo Code)를 사용하여 알고리즘을 기술하는 방법을 연습한다.
3. 가상 코드로 기술된 알고리즘을 C언어로 변환하고 실습을 통하여 프로그램 작성의 기초 원리를 이해한다.

자료구조 및 알고리즘

- 알고리즘 : 특정한 일을 수행하는 명령어들의 유한 집합
- 자료구조 : 주어진 문제를 해결하기 위해 필요한 데이터의 조직화
- 프로그램 : 프로그래밍 언어를 사용한 알고리즘의 구현

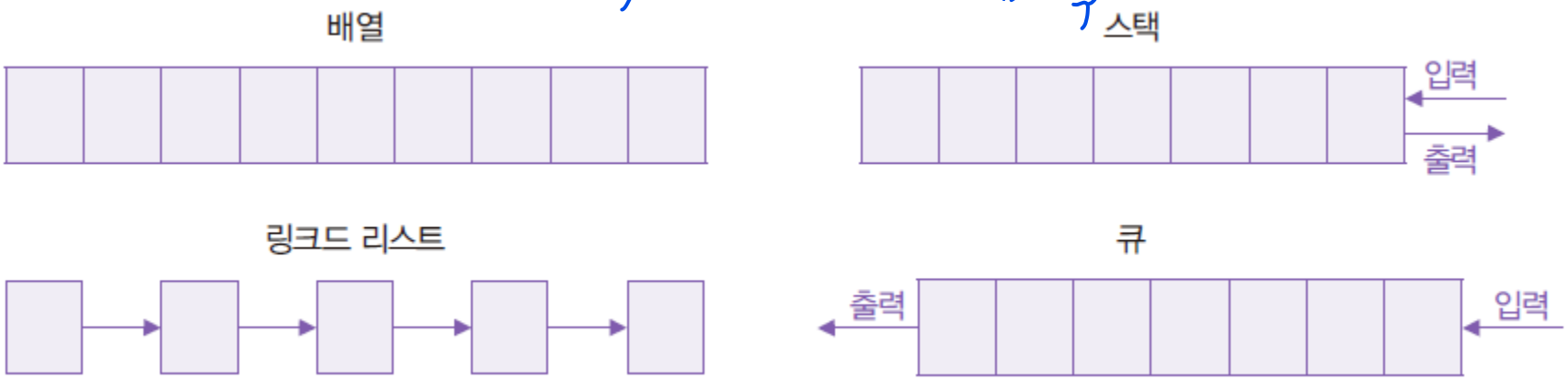
자료구조


자료구조
단순(Primitive)
int
long
double
...
복합(Non-Primitive)
선형(Linear)
배열
링크드 리스트
스택
큐
힙
비선형(Non-Linear)
트리
그래프

자료구조

- 선형 자료구조(Linear Data Structure)

단계별 순차적 탐색 → 모든데이터탐색



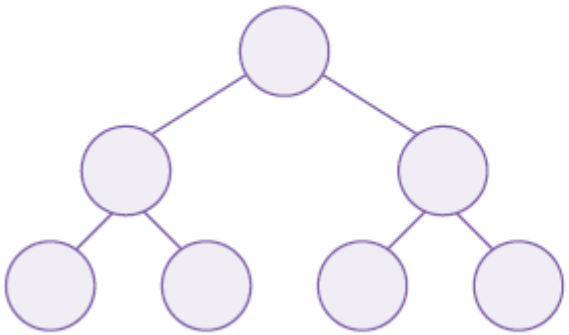
자료구조

- 비선형 자료구조

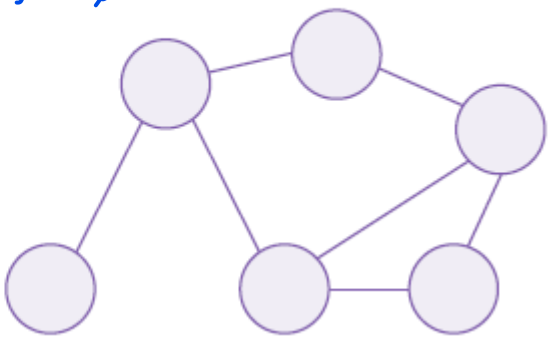
comment -> 다르게

모든 데이터 탐색 ↓ 힘듦

트리

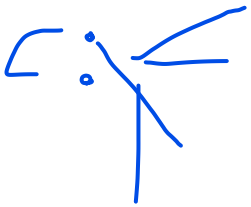


그래프



ex)

파일탐색기

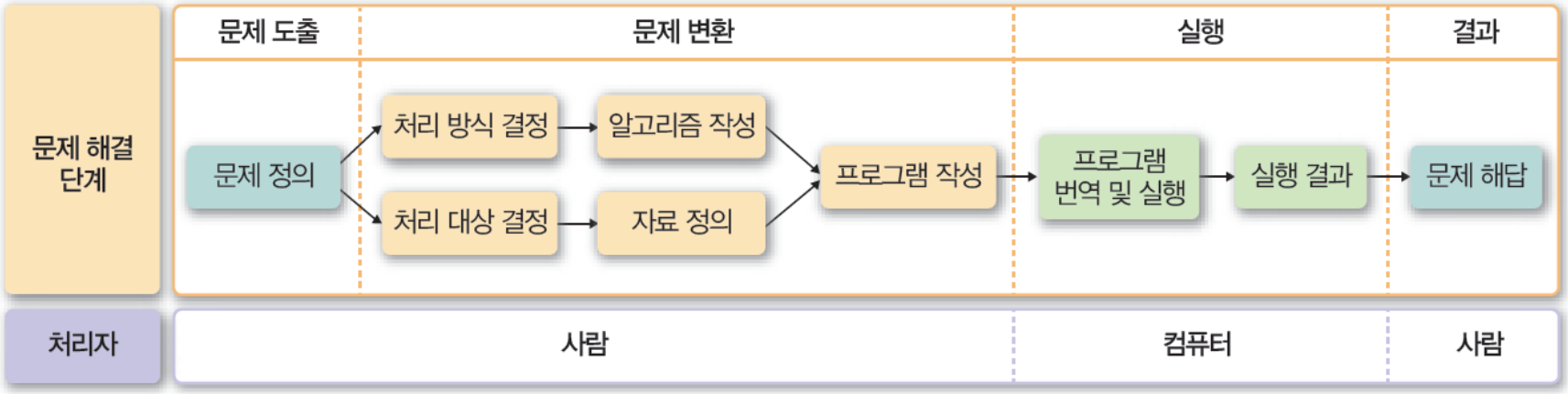


폴더구조

ex

최단거리
네비게이션

알고리즘


문제 해결 단계
문제 도출
문제 변환
실행
결과
문제 정의
처리 방식 결정
알고리즘 작성
처리 대상 결정
자료 정의
프로그램 작성
프로그램 번역 및 실행
실행 결과
문제 해답
처리자
사람
컴퓨터
사람

알고리즘

- 알고리즘
  - 문제해결 방법을 추상화하여 단계적 절차를 논리적으로 기술해 놓은 명세서

- 알고리즘의 조건
  - 입력input : 알고리즘 수행에 필요한 자료가 외부에서 입력으로 제공될 수 있어야 한다.
  - 출력output : 알고리즘 수행 후 하나 이상의 결과를 출력해야 한다.
  - 명확성definiteness : 수행할 작업의 내용과 순서를 나타내는 알고리즘의 명령어들은 명확하게 명세되어야 한다.
  - 유한성finiteness : 알고리즘은 수행 뒤에 반드시 종료되어야 한다.
  - 효과성effectiveness : 알고리즘의 모든 명령어들은 기본적이며 실행이 가능해야 한다.

알고리즘을 기술하는 방법

알고리즘의 표현 방법의 종류

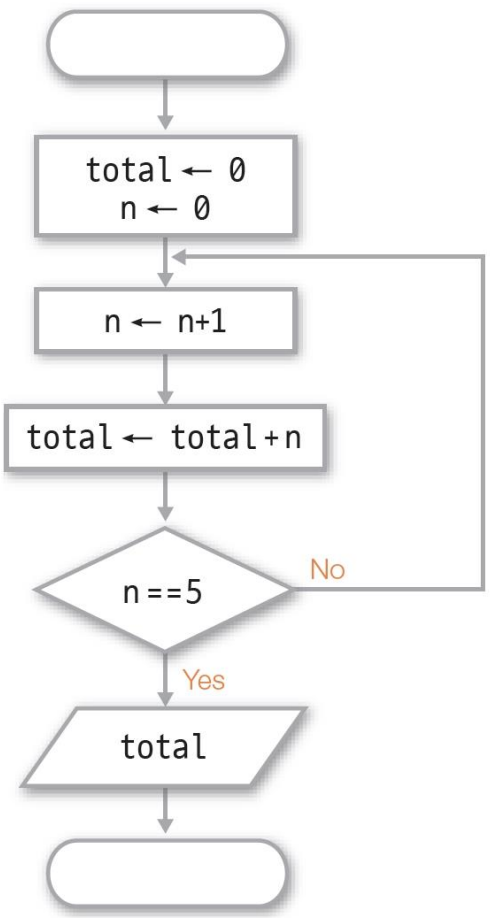
자연어를 이용한 서술적 표현 방법
순서도(Flow chart)를 이용한 도식화 표현 방법
프로그래밍 언어를 이용한 구체화 방법
가상코드(Pseudo-code)를 이용한 추상화 방법

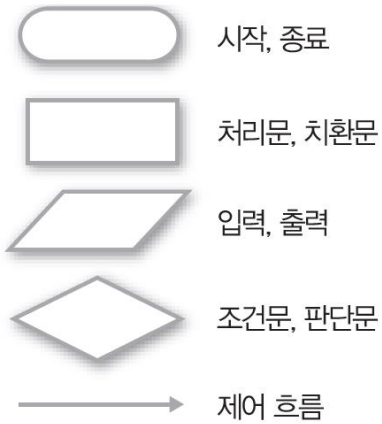
자연언어 기술 예

Find_Max(A, n)

1. 배열 A의 첫번째 값을 tmp에 저장
2. 배열의 A의 다음 값들을 현재 tmp와 비교하여 더 크면 tmp에 저장
3. 2의 과정이 완료되면 tmp 값을 반환

순서도


total ← 0
n ← 0
n ← n+1
total ← total+n
n==5
No
Yes
total


시작, 종료
처리문, 치환문
입력, 출력
조건문, 판단문
제어 흐름

Pseudo-Code

- 가상코드의 형식
  - 기본 요소
    - 기호
      - 변수, 자료형 이름, 프로그램 이름, 레코드 필드 명, 문장의 레이블 등을 나타냄
      - 문자나 숫자의 조합. 첫 문자는 반드시 영문자 사용.
    - 자료형
      - 정수형과 실수형의 수치 자료형, 문자형, 논리형, 포인터, 문자열 등의 모든 자료형 사용
    - 연산자
      - 산술연산자, 관계연산자, 논리연산자

Pseudo-Code 기술 예

```
Algorithm Find_Max(A, n)
    Input : n개의 정수 값이 저장된 배열 A
    Output : A에서 가장 큰 값
    tmp <- A[0]
    for i <- 1 to n do
       if tmp < A[i] then
            tmp <- A[i]
    return tmp
```

가상 코드의 규칙

1 **함수의 선언**

Algorithm name(인수1, 인수2, ...)

여기서 name은 함수의 이름을 의미하고 괄호 안의 인수는 함수 호출에 필요한 매개변수들을 의미한다.

2 **수식의 표현**

- 수식의 산술 연산자들은 일반적인 수학 기호들을 그대로 사용한다.

- 배정문을 위해 “<-” 기호를 사용한다.- 동일 관계의 비교를 위해 “=” 기호를 사용한다.

가상 코드의 규칙

3 **구조적 표현**

- 조건의 비교를 위해 아래와 같은 if 문장 구조를 사용한다.

if

- 반복되는 블록을 실행하기 위해 아래 문장 중의 하나를 사용한다.

while .... do

for .... Do

4 **배열 표현**

1차원 : name[i], 2차원 : name[i,j], 3차원 : name[i,j,k]

5 **함수 호출**

함수명(인수1, 인수2, ...)

가상 코드의 규칙

6 **결과 반환**

- 최종 결과를 반환

return

기본예제

문제 : n개의 정수 값들이 1차원 배열에 주어질 경우 이들 값들 중에서 가장 큰 값을 찾는 알고리즘을 가상코드로 기술한다.

알고리즘 기술

```
Algorithm Find_Max(A,n)
    Input : n개의 정수 값이 저장된 1차원 배열 A
    Output : 배열 A에서 가장 큰 값
    max <- A[0]
    for i <- 1 to n-1 do
        if max < A[i]
            max <- A[i]
    return max
```

기본예제

문제 : n개의 정수 값들이 주어질 경우 이들 중에서 짝수 값들의 합과 홀수 값들의 합을 계산하여 출력하는 알고리즘을 가상코드로 기술한다.

알고리즘 기술

```
Algorithm EvenOdd(A, n)
Input: n개의 양의 정수 값을 갖는 1차원 배열 A
Output: 짝수 값들 합, 홀수 값들의 합
index <- 0
Evensum <- 0
Oddsum <- 0
while (index < n) do {
  if (A[index] is even)
      Evensum <- Evensum + A[index]
  else
      Oddsum <- Oddsum + A[index]
  index <- index + 1
}
print Evensum, Oddsum
end
```

연습예제

문제 : 2차원 n×n 행렬이 주어질 경우 이 행렬의 전치 행렬을 구하는 알고리즘을 가상코드로 기술한다.

(1) 입력: 정수 값들로 구성된 2차원 n×n 행렬이 배열 A에 저장되어 있다고 가정한다. A[i, j]는 2차원 행렬에서 i번째 행과 j번째 열의 값이 저장된 배열의 원소를 의미한다.
(2) 출력: 입력으로 주어진 2차원 행렬 A의 전치 행렬을 계산하여 2차원 행렬 B에 저장한다. 전치 행렬은 행과 열의 값이 뒤 바뀐 행렬을 의미한다.

알고리즘 분석

좋은 알고리즘 이란?

- 수행시간이 적음
- 사용된 메모리 공간이 적음

알고리즘의 평가 기준

- 공간복잡도(Space Complexity) : 알고리즘 수행에 필요한 메모리 공간
- 시간복잡도(Time Complexity) : 알고리즘의 수행 시간

공간복잡도 분석

공간복잡도 : 고정공간 + 가변공간

고정공간 : 문제의 인스탄스(입출력 크기)에 무관한 일정한 양의 메모리 공간

가변공간 : 문제의 인스탄스에 따라 가변적인 메모리 공간

공간복잡도 분석 예

n개의 실수 값들의 합 계산

```
float sum(float list[], int n) {
    float tempsum = 0;
    int i;
    for(i = 0; i < n; i++)
        tempsum += list[i];
    return tempsum; }
```

고정공간 : i, n, tempsum

가변공간 : list[] (call by value or call by reference)

총 공간의 양 : 2*sizeof(int) + sizeof(float) + ?

시간복잡도 분석

분석 방법

- 실험적 수행시간 측정(machine-dependent estimation)
- 알고리즘에서 수행되는 문장들의 스텝 수를 계산하여 측정(machine-independent estimation)

실험적 수행시간 측정 – 사용하지 않음

- 동일한 HW 및 SW 사용
- 알고리즘의 구현 및 테스트 필요
- 제한된 입력에 의한 실험

시간복잡도 분석(실험적 측정 예)

```
void main( void )
{
   clock_t start, finish;
   double  duration;
   start = clock();
    // 수행시간을 측정하고 하는 코드....
     // ....
   finish = clock();
   duration = (double)(finish - start) / CLOCKS_PER_SEC;
   printf("%f 초입니다.₩n", duration);
}
```

시간복잡도 분석(스텝 수)

프로그램 구현 없이 알고리즘에서 문장이 수행되는 스텝 수를 계산하여 분석

스텝 수의 계산 방법

s/e(steps/execution) : 각 문장에 대한 스텝 수 계산

Frequency : 각 문장이 실행되는 빈도 계산

각 문장에 대한 총 스텝 수

(s/e) x (frequency)

예제 #1

- n개의 실수 값들의 합을 계산하는 함수

Statement	s/e	Frequency	Total steps
float sum(float list[],int n)	0	0	0
{	0	0	0
float tempsum=0;	1	1	1
int i;	0	0	0
for(i=0;i<n;i++)	1	n+1	n+1
tempsum+=list[i];	1	n	n
return tempsum;	1	1	1
}	0	0	0
total			2n+3

예제 #2

- n개의 실수 값들의 합을 계산하는 순환 함수

Statement	s/e	Frequency	Total steps
float rsum(float list[],int n) {	0	0	0
if(n)	1	n+1	n+1
return rsum(list,n-1)+list[n-1];	1	n	n
return list[0];	1	1	1
}	0	0	0
Total			2n+2

예제 #3

- m x n 행렬의 덧셈을 계산하는 함수

Statement	s/e Frequency Total steps

void add(int a[][M_SIZE] …)
{
int i,j;
for(i=0;i<m;i++)
for(j=0;j<n;j++)
c[i][j] = a[i][j] + b[i][j];
}	

total	

점근적 표기법(Asymptotic Notation)

정확한 단계 수를 대신하여 수학적 기호를 사용하여 표현

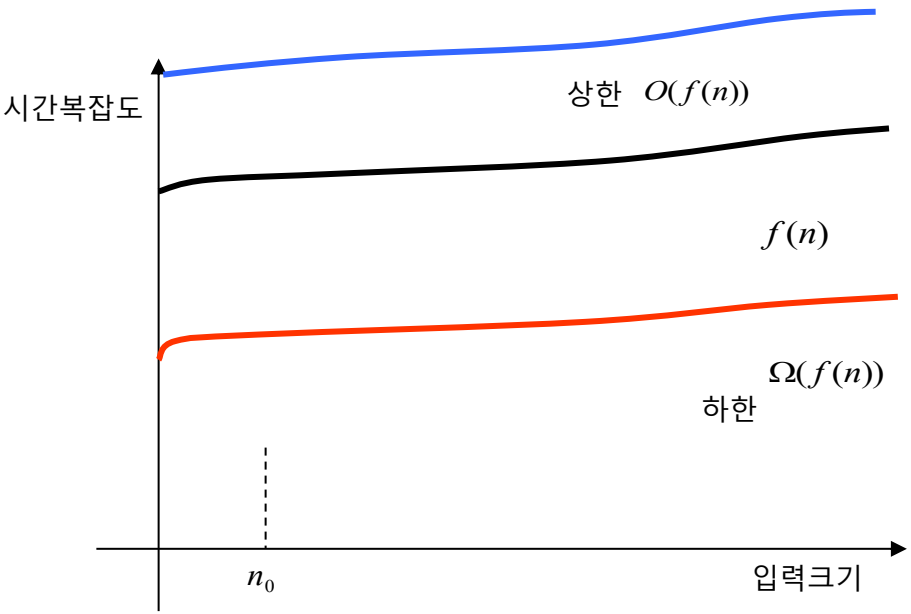
알고리즘의 입력 크기 변화에 따른 수행 시간 예측

동일 문제에 대한 다른 알고리즘 비교

표기 방법

- Big-oh (O) : 상한 표현
- Omega (Ω) : 하한 표현
- Theta (θ) : 상한-하한 표현

점근적 표기법(Asymptotic Notation)



입력 크기에 따른 시간복잡도

시간 복잡도	입력 크기 n					
	1	2	4	8	16	32
1	1	1	1	1	1	1
logn	0	1	2	3	4	5
n	1	2	4	8	16	32
nlogn	0	2	8	24	64	160
n^2	1	4	16	64	256	1024
n^3	1	8	64	512	4096	32768
2^n	2	4	16	256	65536	4294967296
n!	1	2	24	40326	20922789888000	26313×10^{33}

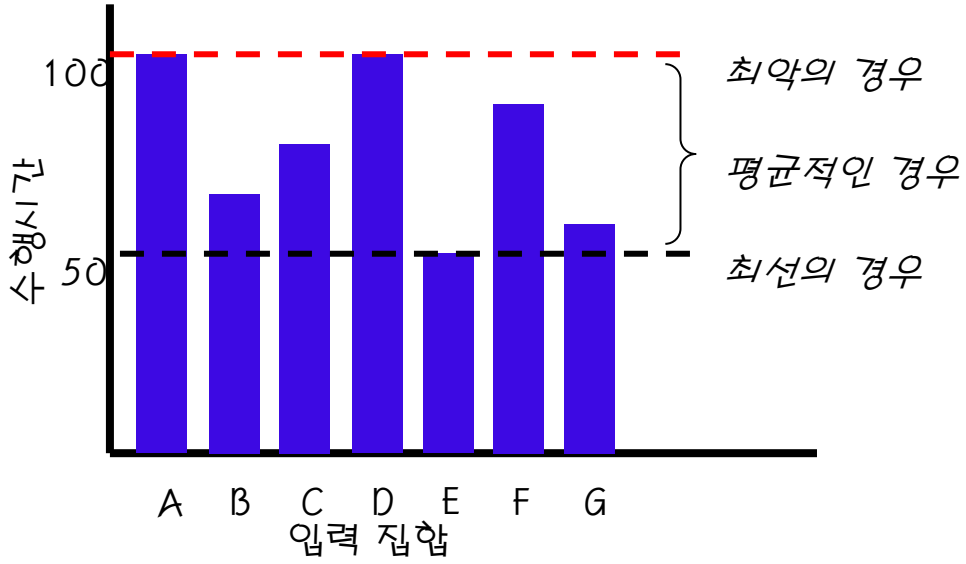
Polynomial Time

Exponential Time

최선, 평균, 최악의 경우

- 알고리즘의 수행시간은 입력 자료 집합에 따라 다를 수 있다.
  (예) 정렬 알고리즘의 수행 시간은 입력 집합에 따라 다를 수 있다.

- **최선의 경우(best case):** 수행 시간이 가장 빠른 경우
- **평균의 경우(average case):** 수행시간이 평균적인 경우
- **최악의 경우(worst case):** 수행 시간이 가장 늦은 경우



최선, 평균, 최악의 경우

- 최선의 경우: 의미가 없는 경우가 많다.
- 평균적인 경우: 계산하기가 상당히 어려움.
- 최악의 경우: 가장 널리 사용된다. 계산하기 쉽고 응용에 따라서 중요한 의미를 가질 수도 있다.
  (예) 비행기 관제업무, 게임, 로보틱스

최선, 평균, 최악의 경우

- (예) 순차탐색
- 최선의 경우: 찾고자 하는 숫자가 맨앞에 있는 경우

  ∴ O(1)

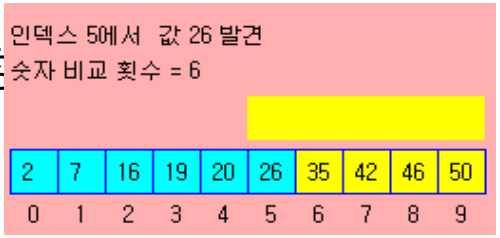
- 최악의 경우: 찾고자 하는 숫자가 맨뒤에 있는 경우

  ∴ O(n)

- 평균적인 경우: 각 요소들이 균일하게 탐색된다고 가정

  $(1+2+\ldots+n)/n=(n+1)/2$

  ∴ O(n)



순환(recursion)이란?

- 알고리즘이나 함수가 수행 도중에 자기 자신을 다시 호출하여 문제를 해결하는 기법

- 정의자체가 순환적으로
  되어 있는 경우에 적합한 방법

팩토리얼 프로그래밍 #1

- 팩토리얼의 정의

$$n! = \begin{cases} 1 & n = 0 \\ n * (n-1)! & n \geq 1 \end{cases}$$

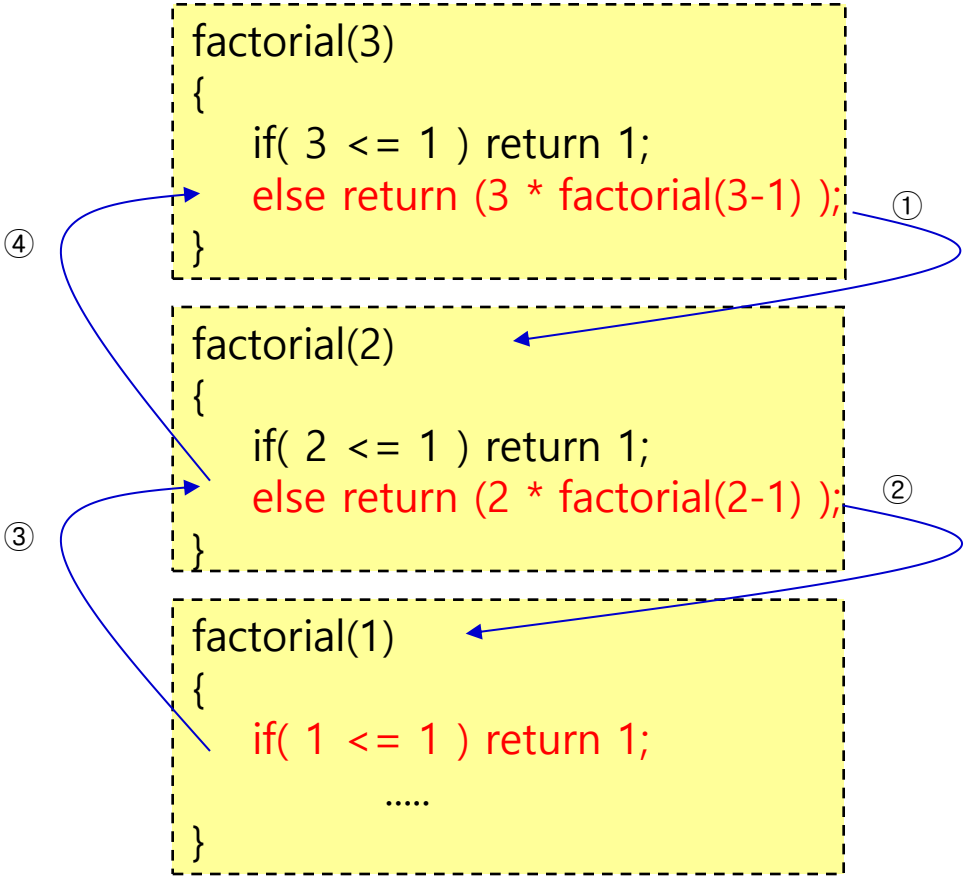
- 팩토리얼 프로그래밍:
  - (n-1)! 팩토리얼을 현재 작성중인 함수를 다시 호출하여 계산(순환 호출)

```
int factorial(int n)
{
   if( n <= 1 ) return(1);
   else return (n * factorial(n-1) );
}
```

순환호출순서

- 팩토리얼 함수의 호출 순서

factorial(5) = 5 * factorial(4)
= 5 * 4 * factorial(3)
= 5 * 4 * 3 * factorial(2)
= 5 * 4 * 3 * 2 * factorial(1)
= 5 * 4 * 3 * 2 * 1
= 120



순환 알고리즘의 구조

- 순환 알고리즘은 다음과 같은 부분들을 포함한다.
  - 순환 호출을 하는 부분
  - 순환 호출을 멈추는 부분

```
int factorial(int n)
{
  if( n <= 1 )  return 1
  else return n * factorial(n-1);
}
```

순환을 멈추는 부분

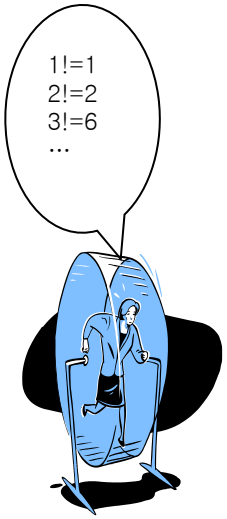
순환 호출을 하는 부분

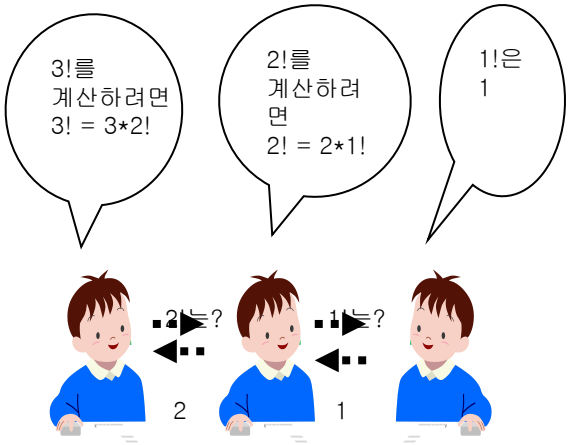
- 만약 순환 호출을 멈추는 부분이 없다면?.
  - 시스템 오류가 발생할 때까지 무한정 호출하게 된다.

순환 <-> 반복

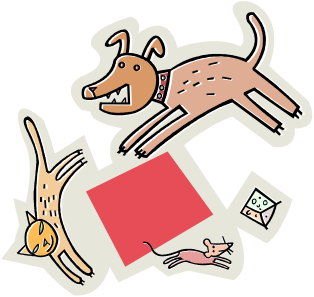
- 컴퓨터에서의 되풀이
  - 순환(recursion): 순환 호출 이용
  - 반복(iteration): for나 while을 이용한 반복
- 대부분의 순환은 반복으로 바꾸어 작성할 수 있다.

- 순환
  - 순환적인 문제에서는 자연스러운 방법
  - 함수 호출의 오버헤드
- 반복
  - 수행속도가 빠르다.
  - 순환적인 문제에 대해서는 프로그램 작성이 아주 어려울 수도 있다.





순환(recursion)의 예



- 팩토리얼 값 구하기

$$n! = \begin{cases} 1 & n = 0 \\ n * (n-1)! & n \geq 1 \end{cases}$$

- 피보나치 수열

$$fib(n) = \begin{cases} 0 & if \quad n = 0 \\ 1 & if \quad n = 1 \\ fib(n-2) + fib(n-1) & otherwise \end{cases}$$

- 이항계수

$${}_nC_k = \begin{cases} 1 & n = 0 \quad or \quad n = k \\ {}_{n-1}C_{k-1} + {}_{n-1}C_k & otherwise \end{cases}$$

- 하노이의 탑
- 이진탐색

실습 문제와 실습 응용에 대한 설명

(실습 문제) n개의 정수 값들이 주어질 경우 이들 값들 중에서 가장 큰 값을 찾기 위한 프로그램을 작성한다. 이 문제를 해결하기 위한 자료구조를 설계하고 알고리즘을 가상 코드로 기술한다.

(응용문제) n개의 양의 정수 값들이 주어질 경우 이들 값들 중에서 짝수 값들의 합과 홀수 값들의 합을 계산한다.